

Benchmark solutions et outils v0

Données et outils IA

ÉTUDE ET EXPÉRIMENTATION DE DISPOSITIFS IA

Référence : 2026-HP-INT-001

Recherche comparative des solutions d'orchestration IA et briques fonctionnelles pouvant être associées (MCP, recherche web...). Elle vise à faire les choix technologiques pour la réalisation des prototypes. Les décisions prises et leur documentation sont détaillées dans [decisions.md](#) (même dossier).

Les tableaux comparatifs ont été produits avec l'IA (Claude Sonnet 5, Anthropic).

CADRAGE FONCTIONNEL ET TECHNIQUE

Cadrage technique

Le cadrage technique s'appuie sur les décisions et priorités du pôle Données. Les solutions retenues devront répondre à ces critères : - open source et gratuits ; - maintenance minimale ; - pouvant regrouper le maximum de fonctionnalités au vu des cas d'usages ; - IA agnostique (n'importe quel fournisseur API peut être utilisé) ; - gestion des utilisateurs possibles ; - connexion possibles à des MCP.

Cadrage fonctionnel

Les neuf cas d'usages présentés dans l'analyse fonctionnelle peuvent être classés en deux catégories : ceux ne nécessitant qu'un agent conversationnel (chatbot), et ceux invitant à travailler en projet.

Pour les cas 1 à 7 (question sur le catalogue, extraction filtrée, traitement Excel ponctuel, enrichissement d'une table, opération géomatique isolée, analyse d'un jeu de données, construction d'un tableau de bord), un chat conversationnel est suffisant, et permet d'utiliser des outils (Excel, QGIS, SQL, recherche sur le web...)

Les cas 8 et 9 (actualisation/reproduction d'une chaîne de traitement et conception méthodo), nécessitent un accès au dossier local du projet, pour permettre à l'IA de consulter son contenu selon ses besoins, de modifier les documents, d'en créer, et de prendre certaines décisions de manière autonome... Ce fonctionnement permet notamment de s'aider de l'IA pour documenter la méthodologie du travail. Ce mode de fonctionnement pourrait aussi convenir pour le cas 7.

Le développement se concentrera d'abord sur les cas d'usage 1 (connaissance des données Audiar / catalogue), 2 (extraction depuis une table en base), 3 (manipulation Excel assistée) et 5 (géomatique/QGIS) de l'analyse fonctionnelle.

Toutefois, le prototype développé pourra si possible de fonctionner en mode projet, avec un accès à un dossier de travail complet.

SOLUTIONS D'ORCHESTRATION

On peut distinguer deux types de solutions d'orchestration, d'un côté les solutions prêtes à l'emploi, qui ne nécessitent que peu d'écriture de code, et de l'autre les frameworks, des briques de code à assembler. Il s'agira d'abord d'opter pour l'une ou l'autre de ces solutions, puis de choisir la technologie la plus appropriée.

Solutions prêtes à l'emploi

TABLEAU COMPARATIF

Recherche effectuée les 17/07/2026 et 20/07/2026.

Les candidats sont répartis en deux tableaux selon leur orientation dominante (cf. ligne « Mode projet ») : d'un côté les solutions orientées chat/RAG documentaire, de l'autre les solutions orientées agent autonome/mode projet — une distinction qui recoupe celle du cadrage fonctionnel présenté plus haut (cas 1 à 7 vs cas 8 et 9).

Solutions orientées chat / RAG documentaire

Critère	AnythingLLM	Open WebUI	LibreChat	Jan	GPT4All
Licence	MIT	"Open WebUI License" (BSD 3-Clause + clause de marque, non certifiée OSI depuis la v0.6.6)	MIT	Apache 2.0	MIT
Repo	Mintplex-Labs/anything-llm	open-webui/open-webui	danny-avila/LibreChat	janhq/jan	nomic-ai/gpt4all
Auteur	Mintplex Labs, startup fondée en 2022 par Timothy Carambat (Irvine, Californie), financée en pré-amorçage par Y Combinator (500k\$, promotion été 2022) – petite structure avec investisseurs en capital-risque, motivation commerciale (offre cloud/entreprise) à surveiller sur la durée.	Projet indépendant fondé par Tim Jaeryang Baek, pas de levée de fonds formelle en capital-risque (soutien ponctuel Mozilla Builders/GitHub Accelerator), financé par sponsoring ; une offre "Open WebUI for Enterprise" existe déjà – signal d'une monétisation en cours, cohérent avec le changement de licence.	Projet personnel de Danny Avila, développeur indépendant (pas de structure commerciale derrière), financé par sponsoring individuel (GitHub Sponsors, ex. Shopify) – risque de "bus factor" : mainteneur unique, en parallèle de son emploi principal.	Jan (janhq) – équipe/société avec contacts business (hello@jan.ai) et recrutement (hr@jan.ai) identifiés, mais taille et financement non confirmés dans les sources consultées.	Nomic AI (société américaine), qui commercialise par ailleurs une plateforme payante distincte (Nomic Atlas, embeddings/visualisation) – schéma "produit phare open source + activité commerciale séparée" déjà en place.

Critère	AnythingLLM	Open WebUI	LibreChat	Jan	GPT4All
Mode projet	Moyen-fort – notion de “workspace” par projet, mode @agent avec permissions dossier accordées explicitement (lecture/écriture) sur le poste local.	Faible-moyen – interface de chat/RAG avec appel d’outils MCP via un proxy (mcpo), pas conçu pour un mode projet autonome multi-étapes façon Cowork.	Faible-moyen – agents avec exécution de code et upload de fichiers, mais orienté conversation multi-utilisateurs plus que dossier de travail persistant façon Cowork.	Faible-moyen – avant tout un client de chat/inférence locale (comme AnythingLLM/Op en WebUI) ; le support MCP est un ajout récent (v0.7.3), pas une conception “mode projet” dès l’origine.	Faible – historiquement un client de chat local avec RAG documentaire (“LocalDocs”) ; l’intégration MCP est récente et peu documentée, le positionnement reste “assistant de bureau” plutôt qu’agent autonome.
Compatibilité MCP	Natif (stdio/SSE/HTTP), agents avec compétences (“skills”) configurables sans code.	Via proxy mcpo (MCP → OpenAPI) ou connexion HTTP/SSE directe ; pas de support MCP natif “de bout en bout”.	Natif (stdio/HTTP/SSE), outils MCP assignables à des agents spécifiques.	Natif depuis la v0.7.3 (juillet 2025) ; connexion à n’importe quel serveur MCP depuis la v0.7.9 (accès fichiers, navigation web, outils personnalisés).	Intégration MCP annoncée dans l’app desktop, mais moins documentée/mature que chez Jan ou les candidats “mode projet”.
Compétences nécessaires	Déploiement simple (Docker ou app Desktop clé en main), accessible à un profil admin système basique ; configuration des agents/skills en no-code ; contribuer/étendre demande du JavaScript (Node.js).	Déploiement via Docker (une commande), accessible à un profil admin système basique ; le mode agent/MCP nécessite de configurer le proxy mcpo (un peu de ligne de commande) ; contribuer demande du Python et du Svelte.	Déploiement via Docker Compose avec plusieurs services à orchestrer (MongoDB, Redis, MeiliSearch, pgvector) – demande une compétence admin système/Docker plus confirmée que les autres candidats ; contribuer demande du JavaScript/TypeScript.	Installation “one-click” (Windows/macOS/Linux, dont Microsoft Store et Flathub) sans compétence dev ; build depuis les sources demande Node.js/Yarn/React (Tauri).	Installation simple sans compétence dev sur Windows/macOS/Linux.

Critère	AnythingLLM	Open WebUI	LibreChat	Jan	GPT4All
Facilité de prise en main	Facile – interface de chat familière type ChatGPT, onboarding guidé (création de workspace, dépôt de documents) ; le mode <code>@agent</code> demande un peu d'appropriation mais reste accessible aux profils intermédiaires.	Facile pour le chat/RAG courant (interface proche de ChatGPT, prise en main immédiate) ; la configuration MCP/agentique reste du ressort de l'administrateur, transparente une fois en place pour l'utilisateur final.	Facile pour l'usage courant (autre clone ChatGPT, prise en main immédiate) ; l'utilisateur final choisit un agent préconfiguré dans une liste, sans avoir à gérer les outils/MCP lui-même.	Facile – interface de chat type ChatGPT avec gestion de modèles locaux intégrée, comparable à AnythingLLM/Op en WebUI.	Facile – l'une des applications de chat local les plus anciennes et abouties dans sa simplicité (250 000+ utilisateurs actifs mensuels revendiqués).
Code	JavaScript/Node.js (serveur Express) + React (frontend Vite)	Python/FastAPI (backend) + Svelte (frontend)	JavaScript/TypeScript (Node/Express + React) + MongoDB	TypeScript (75%) + Rust (Tauri, 20%) + Swift (app macOS) + Python.	Historiquement C++ (moteur d'inférence) + bindings Python/JS (à reconformer pour la version actuelle).
Souveraineté / choix du fournisseur	Bon – moteur d'inférence embarqué (llama.cpp), Ollama, ou API cloud au choix.	Bon – Ollama natif + tout endpoint compatible API OpenAI.	Bon – liste très large de fournisseurs supportés.	Excellent – conçu pour tourner 100% hors-ligne (modèles locaux via llama.cpp), ponts optionnels vers API cloud (OpenAI, Anthropic, Mistral, Groq, MiniMax).	Excellent – 100% local par conception, aucune dépendance cloud requise.
Multi-utilisateurs et gestion des droits	Oui en self-hosted (Docker) : rôles admin/manager/user ; l'app Desktop reste mono-utilisateur.	Natif et mature – RBAC, groupes, SSO possible ; un des meilleurs du comparatif sur ce point.	Natif et solide dès la conception – comptes, rôles, auth complète, pensé pour un déploiement d'équipe.	Non – application de bureau mono-utilisateur, comme AnythingLLM Desktop.	Non – application de bureau mono-utilisateur.
Traçabilité / journal des opérations	Historique des conversations par workspace ; pas de journal d'audit détaillé des actions d'agent (<code>@agent</code>).	Logs d'utilisation côté serveur ; détail des appels d'outils dépendant du proxy <code>mcpo</code> , moins visible nativement qu'un IDE.	Historique des conversations en base (MongoDB) et recherche full-text (MeiliSearch), mais pas de "journal d'audit" spécifique aux actions d'agent.	Non documentée pour un usage de type audit d'agent (application de chat, pas de journal d'audit formalisé).	Non documentée pour un usage de type audit d'agent (application de chat, pas d'agent multi-étapes formalisé).

Critère	AnythingLLM	Open WebUI	LibreChat	Jan	GPT4All
Infrastructure requise et estimation de coût	Desktop = gratuit sur poste local ; version Docker self-hosted nécessite un serveur (RAM significative si inférence embarquée utilisée) – coût modéré.	Nécessite un serveur (Docker), potentiellement GPU si modèles locaux lourds – coût d'hébergement à prévoir en continu.	Infra la plus lourde du comparatif : plusieurs services à héberger (MongoDB, Redis, MeiliSearch, pgvector) – coût de maintenance/hébergement plus élevé.	Aucune infra serveur – poste local, gratuit hors coût API cloud optionnel ; 8 à 32 Go de RAM recommandés selon la taille des modèles locaux.	Aucune infra serveur, gratuit, tourne sur poste local.
Compatibilité Windows	Oui – app Desktop Windows/macOS /Linux.	Oui, via Docker Desktop (WSL2).	Oui, via Docker Desktop (WSL2) uniquement – pas d'app native Windows.	Oui, nativement (dont distribution officielle via Microsoft Store).	Oui, nativement.
Persistance mémoire/contexte	Bonne – RAG intégré avec embeddings stockés par workspace (mémoire documentaire persistante) + historique conversationnel par fil.	Historique des conversations persistant côté serveur, RAG documentaire.	Robuste – base dédiée (MongoDB), conçu pour la persistance dès le départ.	Historique de conversation local ; pas de mémoire documentaire/RAG aussi structurée qu'AnythingLLM à notre connaissance.	"LocalDocs" (RAG sur fichiers locaux) pour la mémoire documentaire ; historique de conversation local.
Richesse de la personnalisation	Mode agent no-code (créateur de "skills" sans code) – bonne richesse pour des profils non-développeurs.	Pipelines/"fonctions" Python + large écosystème de plugins communautaires.	Agents configurables et actions personnalisées, mais demande du code pour aller loin.	"Assistants personnalisés" configurables + serveur API compatible OpenAI local (localhost:1337) réutilisable par d'autres applications.	Plus limitée sur l'aspect agentique – personnalisation portant surtout sur le choix des modèles et les sources LocalDocs.

Critère	AnythingLLM	Open WebUI	LibreChat	Jan	GPT4All
Pérennité de la licence gratuite/open source	Moyen – MIT, mais startup à capital-risque (YC) sous pression de monétisation ; le code déjà publié reste MIT, mais risque que les futures fonctionnalités avancées basculent vers une offre cloud/entreprise payante plutôt que d'enrichir le dépôt open source.	Faible-moyen – précédent concret défavorable : passage en 2025 d'une licence permissive à la "Open WebUI License" (non-OSI), au moment même où une offre Enterprise apparaissait ; tendance déjà amorcée vers plus de restrictions.	Élevé sur la licence (MIT, pas de capital-risque poussant à la fermeture) mais incertitude sur la continuité de la maintenance (mainteneur unique) – distinguer "reste gratuit/ouvert" (probable) de "reste activement maintenu" (moins certain).	Moyen – Apache 2.0, mais modèle économique de la société (Jan/janhq) non documenté publiquement ; pas de signal négatif concret à ce jour, mais pas de garantie de fondation neutre non plus.	Moyen – MIT sans ambiguïté, mais société avec activité commerciale par ailleurs (Nomic Atlas) ; pas de signal négatif concret sur GPT4All spécifiquement à ce jour.
Capacité de la communauté à maintenir un fork en cas de fermeture	Moyen – MIT et large communauté (54k ★) rendent un fork techniquement facile, mais aucune crise de licence ne s'est encore produite pour tester cette hypothèse en pratique ; à surveiller si Mintplex Labs venait à restreindre l'accès aux futures fonctionnalités.	Cas déjà documenté – le changement de licence d'avril 2025 a réellement déclenché des discussions actives de fork (repartant de la dernière version BSD), la communauté citant elle-même les précédents réussis de Valkey (fork de Redis) et OpenSearch (fork d'Elasticsearch par AWS) ; cela dit, à ce jour aucun fork dominant et pérenne d'Open WebUI ne s'est encore imposé – des discussions, pas encore un projet abouti.	Moyen-élevé – MIT, projet important (33k ★) ; le risque principal ici est l'abandon plutôt qu'une fermeture de licence (pas de capital-risque poussant à restreindre), et les forks de projets abandonnés sont un schéma courant et généralement viable pour un dépôt de cette taille.	Moyen – Apache 2.0 permissif, communauté conséquente (43,6k ★), mais aucune crise de licence n'a eu lieu pour tester cette hypothèse.	Moyen-élevé – MIT, la plus grande communauté de tous les candidats de ce document (77,4k ★), mais aucune crise testée en pratique.

Critère	AnythingLLM	Open WebUI	LibreChat	Jan	GPT4All
Avantages	App Desktop clé en main, moteur d'inférence embarqué ; ~54k ★ ; RAG intégré nativement ; workspaces isolés par projet.	Excellent pour RAG/catalogue documentaire ; large communauté ; déploiement Docker simple – bon point souveraineté.	~33k ★ ; réellement multi-utilisateurs avec auth ; licence MIT sans ambiguïté ; bonne intégration MCP.	Souveraineté/local-first abouti ; large communauté (43,6k ★) ; support MCP fonctionnel ; distribution Windows Store officielle.	Client de chat local le plus ancien et éprouvé ; plus grande communauté du comparatif (77,4k ★) ; MIT sans ambiguïté ; RAG local mature (LocalDocs).
Inconvénients	Mode agent (fichiers, MCP) moins mature que chez les spécialistes ; permissions dossier globales, granularité grossière.	Licence non certifiée OSI depuis 2025 – point de vigilance si politique stricte sur les licences ; mode agentique nécessite mcpo.	Pensé comme clone de ChatGPT plutôt qu'agent autonome ; gouvernance mono-mainteneur, moins résiliente.	Reste avant tout un client de chat/inférence locale ; support MCP récent et peu éprouvé ; pas de multi-utilisateurs.	Le moins orienté agent de tous les candidats du tableau ; pas de multi-utilisateurs ni de contrôle pas à pas.

Solutions orientées agent autonome / mode projet

Critère	Goose	VS Code + Cline	Eigent	Open Cowork	OpenWork	OpenHands
Licence	Apache 2.0	VS Code : MIT (cœur) ; Cline : Apache 2.0	Apache 2.0	MIT	MIT	MIT (à reconformer précisément)
Repo	block/goose	cline/cline	eigent-ai/eigent	OpenCoworkAI/open-cowork	different-ai/openwork	OpenHands/OpenHands

Critère	Goose	VS Code + Cline	Eigent	Open Cowork	OpenWork	OpenHands
Auteur	Créé par Block, Inc. (maison mère de Square/Cash App) ; gouvernance transférée à la Linux Foundation (Agentic AI Foundation) en décembre 2025 – gouvernance de fondation neutre, réduit la dépendance à un éditeur commercial unique.	VS Code = Microsoft (éditeur majeur, très pérenne). Cline = Cline Bot Inc., startup fondée par Saoud Rizwan, 32M\$ levés (amorçage + série A menée par Emergence Capital, valorisation ~110M\$) – structure bien financée mais avec pression investisseurs à générer du revenu (offre entreprise déjà évoquée).	Eigent AI, startup fondée en 2023, basée à Londres (~8 employés), investie par Sky Arc Capital – structure jeune et petite. S'appuie sur le framework de recherche ouvert CAMEL-AI (collectif de recherche distinct d'Eigent AI, à ne pas confondre) comme brique multi-agents sous-jacente.	Organisation GitHub "OpenCowork AI", membres non rendus publics – aucune société ni levée de fonds identifiée. Transparence faible sur qui est derrière le projet, mais pas de pression investisseur identifiée non plus (à double tranchant).	different-ai / OpenWork Labs (contact ben@openworklabs.com) – structure commerciale assumée dès aujourd'hui : plan Entreprise payant (SSO, SLA, LTS) et accès Windows lui-même payant existent déjà, ce n'est pas qu'un risque futur comme chez les autres candidats.	Société OpenHands (ex-All Hands AI), 18,8M\$ levés en série A ; offres commerciales "OpenHands Cloud"/"Open Hands Enterprise" actives en parallèle du projet open source.
Mode projet	Fort – agent autonome CLI + Desktop, exécute commandes shell, édite des fichiers, orchestre des tâches multi-étapes sans supervision pas à pas, sur le modèle direct de Claude Code/Cowork.	Fort – agent qui lit/édite les fichiers du dossier ouvert, exécute des commandes, avec approbation explicite de chaque action (mode Plan puis Act), diff affiché avant application.	Très fort – explicitement conçu comme alternative locale et open source à Claude Cowork ; workforce multi-agents exécutant des workflows complexes en parallèle sur le poste.	Très fort – clone assumé de Claude Cowork : enveloppe Claude Code/autres CLI dans une app Desktop, isolation sandbox (WSL2 sous Windows, Lima sous macOS), 4 modes par tâche (Autonome, Collaboratif, Multi-LLM, "Think With Me").	Très fort – se présente explicitement comme "l'alternative open source à Claude Cowork/Codex" ; mode Host (exécution locale) ou Client (connexion à un serveur OpenCode distant), plan d'exécution affiché comme une timeline, "ejectable" (réutilisable sans l'interface).	Fort mais positionné différemment – repositionné en 2026 comme "Agent Canvas", un centre de contrôle self-hosted pour orchestrer plusieurs agents de développement (lui-même, mais aussi Claude Code, Codex, Gemini via le protocole ACP), en local, Docker, VM ou cloud, avec automatisations programmées (Slack/GitHub/Linear).

Critère	Goose	VS Code + Cline	Eigent	Open Cowork	OpenWork	OpenHands
Compatibilité MCP	Natif, extensions MCP dédiées (>3000 serveurs communautaires référencés).	Natif et mature, marketplace de serveurs MCP intégré à l'extension.	Natif, plus de 200 outils MCP embarqués, ajout possible de serveurs MCP supplémentaires.	Natif annoncé (navigateurs, Notion, apps desktop).	Le moteur sous-jacent (OpenCode) utilise surtout un système de plugins propre ; support MCP à confirmer précisément.	Oui, intégration MCP présente parmi les outils de l'agent.
Compétences nécessaires	Installation/usage de base sans compétence dev (binaire Desktop ou script d'installation) ; personnalisation avancée (recettes) via YAML, pas de code obligatoire ; contribuer au cœur demande du Rust.	Installation simple (extension VS Code standard), aucune compétence dev requise pour l'usage de base ; mais l'usage quotidien se fait dans un éditeur de code (arborescence de fichiers, terminal visibles) – suppose un minimum de culture technique côté utilisateur ; contribuer à Cline demande du TypeScript.	Déploiement via app Desktop (installateur) ou Docker, accessible sans compétence dev pour l'usage de base ; configurer plus finement l'orchestration multi-agents (rôles, parallélisation) demande davantage de compétence côté administration ; contribuer demande du Python et du TypeScript.	Installation "one-click" annoncée sans compétence dev ; sandboxing (WSL2/Lima) géré automatiquement par l'outil.	App Desktop simple à utiliser (téléchargement direct macOS/Linux) ; toute personnalisation poussée ou build depuis les sources demande Node.js/pnpm /Rust (Tauri)/Bun – plus technique que la plupart des autres candidats.	Installation via npm (agent-canvas) ou Docker ; le mode "sans sandbox" donne un accès complet au système de fichiers (avertissement explicite du projet) – configuration à soigner, plus technique que les candidats Desktop clé en main.

Critère	Goose	VS Code + Cline	Eigent	Open Cowork	OpenWork	OpenHands
Facilité de prise en main	Moyenne à difficile – usage par défaut en ligne de commande (CLI), peu naturel pour un non-développeur ; le Desktop app simplifie l'accès mais reste un outil "assistant technique" sans onboarding guidé ; convient surtout aux profils avancés.	Difficile pour un profil généraliste – nécessite de se familiariser avec un éditeur de code (arborescence, terminal, panneau de diff) avant même d'utiliser Cline ; plus naturel pour les profils avancés déjà à l'aise avec un IDE.	Moyenne – interface Desktop pensée pour l'utilisateur final (à la Cowork), mais le concept de "workforce" multi-agents (assigner des rôles, superviser plusieurs agents en parallèle) est nouveau et demande un temps d'appropriation supérieur à un simple chat ; documentation/community encore limitée vu la jeunesse du projet.	Probablement facile à moyenne (positionnement "one-click", pas de code requis) mais projet jeune et peu documenté publiquement en dehors du dépôt – retour d'expérience réel manquant pour confirmer.	Bonne pour l'usage de base (app Desktop macOS/Linux, interface pensée pour être moins "développeur" que les outils OpenCode bruts – c'est l'objectif affiché du projet) ; complications dès qu'on sort du cas simple (Windows payant, cf. plus bas).	Difficile pour un profil non-développeur – vocabulaire et documentation clairement orientés ingénierie logicielle (agents de code, CI/CD, intégrations GitHub).
Code	Rust (cœur) + TypeScript (interface Desktop Electron)	TypeScript (extension + interface webview React)	Python (backend, s'appuie sur CAMEL-AI) + TypeScript (frontend)	Non détaillé précisément dans les sources consultées (app desktop multiplatforme).	TypeScript (84%) + JavaScript + Rust (Tauri, app desktop) + CSS.	Python (65%) + TypeScript (33%).
Souveraineté / choix du fournisseur	Excellent – 15 à 25+ fournisseurs supportés dont modèles locaux via Ollama ; aucun verrouillage.	Excellent – quasiment tous les fournisseurs dont le local (Ollama).	Bon – revendiqué "model-agnostique" (Claude, GPT, Gemini, LLM local).	Bon – multi-modèle annoncé (Claude, GPT, Gemini, Kimi, GLM, Ollama).	Bon en théorie (moteur OpenCode multi-fournisseurs) ; local par défaut (lié à 127.0.0.1).	Bon – "bring your own model", self-hosted par défaut, mais une offre cloud propriétaire (OpenHands Cloud) existe aussi et n'est pas souveraine si utilisée.

Critère	Goose	VS Code + Cline	Eigent	Open Cowork	OpenWork	OpenHands
Multi-utilisateurs et gestion des droits	Aucun – outil mono-utilisateur par conception (poste local, CLI/Desktop).	Aucun – extension locale d'un IDE personnel, pas de notion de comptes.	SSO/RBAC/au dit annoncés comme inclus, mais projet jeune : maturité réelle non vérifiée en usage.	Non documenté – semble pensé pour un usage individuel, avec pilotage à distance via Feishu/Slack plutôt qu'un vrai système de comptes/rôles .	Pas de multi-utilisateurs natif dans la version gratuite ; mode "Client" pour se connecter à un serveur partagé, mais SSO/équipe réservés au plan Entreprise payant.	Partiel – un serveur d'agents peut être partagé par une équipe technique, mais pas de gestion de droits façon LibreChat/Open WebUI pour des profils métier variés.
Traçabilité / journal des opérations	Historique de session local (CLI/Desktop) et "recipes" YAML consultables, mais pas de journal d'audit centralisé formalisé.	Chaque action nécessite une approbation explicite avec diff visible (mode Plan/Act) ; pas de journal d'audit centralisé exportable en tant que tel, mais traçabilité immédiate au fil de l'eau.	Journaux d'audit explicitement mis en avant par l'éditeur (fonction "entreprise") – bon point sur le papier, non vérifié en usage réel vu la jeunesse du projet.	Non documenté précisément dans les sources consultées – à vérifier.	Point fort explicite et revendiqué du projet : "Auditable – show what happened, when, and why" ; permissions demandées action par action (allow once/always/deny) ; export d'un rapport de debug complet.	Bonne dans l'esprit développeur (logs d'exécution, intégration CI/GitHub), mais pas formalisée comme un "journal d'audit" lisible par un profil non-développeur.
Infrastructure requise et estimation de coût	Aucune infra serveur – tourne sur le poste (CLI/Desktop) ; coût nul hors API modèle.	Aucune infra serveur – tourne sur le poste ; gratuit hors coût API modèle.	App Desktop locale ou Docker – léger sur poste local.	Aucune infra serveur – poste local avec sandbox WSL2/Lima ; gratuit hors coût API modèle.	App Desktop gratuite sous macOS/Linux ; le mode Client vers un serveur partagé et le plan Entreprise ont un coût d'infra/abonnement à part.	Variable – gratuit sur poste local à coûteux en continu (VM/cloud) ; offre commerciale disponible si besoin de fiabilité accrue.

Critère	Goose	VS Code + Cline	Eigent	Open Cowork	OpenWork	OpenHands
Compatibilité Windows	Oui – CLI et Desktop (Electron) disponibles nativement sous Windows.	Oui, nativement (VS Code fonctionne parfaitement sous Windows).	À confirmer – app Desktop généralement multiplateforme, mais non vérifié précisément pour Windows.	Oui, explicitement supporté (sandbox via WSL2 sous Windows).	Payant – l'accès Windows n'est disponible que via un plan de support payant (openworklabs.com/pricing#windows-support) ; macOS et Linux seuls sont gratuits en téléchargement direct. Point bloquant concret pour l'Audiar, dont l'environnement est très majoritairement Windows.	Oui via Docker Desktop (guide dédié README.windows.md fourni par le projet).
Persistance mémoire/contexte	Sessions et "recipies" réutilisables stockées localement ; pas de mémoire long terme structurée nativement (pas de base vectorielle intégrée).	Contexte de tâche lié à la session VS Code (un "Plan" peut être sauvegardé), pas de mémoire long terme structurée nativement.	État de tâche partagé entre agents de la "workforce", mémoire long terme peu documentée (projet jeune).	Non documenté précisément dans les sources consultées.	Sessions et "templates" de workflows réutilisables, sauvegardés localement.	Sessions et automatisations programmées (webhooks, cron), orientées suivi de tâches de développement plutôt que mémoire documentaire.
Richesse de la personnalisation	Extensions MCP + "recipies" YAML partageables – bonne richesse, nécessite l'édition de fichiers de config.	Marketplace MCP + règles de projet (.clinerules) – bonne richesse, orientée développeur.	Architecture multi-agents personnalisable (rôles) + 200 outils MCP – riche mais configuration plus complexe.	Système de "Skills" générant directement des documents bureautiques (PPTX/DOCX/XLSX/PDF) + outils MCP – bonne richesse, utile de fait pour le cas d'usage 3 (Excel).	Gestionnaire de "skills" (dossiers .opencode/skills) + plugins OpenCode – riche mais orientée développeur (édition de fichiers JSON/config).	Très riche mais orientée développeur – automatisations, intégrations tierces (Slack, GitHub, Linear, Notion), choix libre de l'agent sous-jacent (protocole ACP).

Critère	Goose	VS Code + Cline	Eigent	Open Cowork	OpenWork	OpenHands
Pérennité de la licence gratuite/open source	Élevé – gouvernance transférée à une fondation neutre (Linux Foundation, déc. 2025), structurellement moins encline à fermer la licence qu'un éditeur unique ; code Apache 2.0 déjà publié, de toute façon irrévocable.	Moyen – VS Code (Microsoft) reste gratuit sans ambiguïté depuis plus de 10 ans, signal de continuité fort ; Cline (32M\$ levés) a la même dynamique qu'AnythingLLM : code Apache 2.0 déjà publié irrévocable, mais risque que les futures fonctionnalités soient réservées à une offre entreprise déjà évoquée.	Faible-moyen – startup jeune (~8 personnes) et financée par capital-risque ; les fonctions "entreprise" (SSO, RBAC, audit) sont présentées comme incluses aujourd'hui, mais à surveiller si elles basculent vers une édition payante à mesure que l'éditeur cherche à générer du revenu ; recul insuffisant (projet pré-v1.0) pour juger de la trajectoire.	Moyen – MIT, mais organisation anonyme (pas de société identifiée derrière) : ni le risque "capital-risque" des autres candidats, ni la garantie d'une fondation neutre. Incertitude par manque d'information plutôt que signal négatif concret.	Faible-moyen – modèle "open-core" déjà pleinement assumé et monétisé aujourd'hui (plan Entreprise payant ET accès Windows payant existent dès maintenant), contrairement aux autres candidats où la monétisation reste un risque futur hypothétique.	Moyen – MIT, mais société bien financée avec deux offres commerciales déjà actives (Cloud/Enterprise) ; le cœur restera probablement ouvert (stratégie open-core classique), mais les fonctionnalités avancées ont vocation à être payantes.

Critère	Goose	VS Code + Cline	Eigent	Open Cowork	OpenWork	OpenHands
Capacité de la communauté à maintenir un fork en cas de fermeture	Élevé – Apache 2.0 permissif, mais surtout risque déclencheur déjà faible (gouvernance en fondation) ; si le cas se présentait, l'écosystème Linux Foundation a un précédent direct et réussi : Valkey, le fork communautaire de Redis créé après le changement de licence de Redis Inc. en 2024, aujourd'hui bien maintenu sous cette même fondation. Bémol : bassin de contributeurs Rust plus restreint que JS/Python, ce qui pourrait ralentir un fork si besoin.	Élevé – le précédent le plus solide du tableau. VS Code dispose déjà d'un fork mature et activement maintenu depuis des années : VSCodium, qui retire la télémétrie et les éléments propriétaires de Microsoft et s'appuie sur le registre d'extensions ouvert Open VSX (Eclipse Foundation) – la preuve concrète que ce schéma fonctionne dans la durée. Côté Cline, l'écosystème a montré une forte propension à forker (Roo Code, puis Kilo Code fusionnant Cline + Roo Code), même si la pérennité de ces forks reste inégale (Roo Code a été archivé en mai 2026).	Faible-moyen – Apache 2.0 permissif en théorie, mais communauté encore jeune et restreinte (14k ★, projet pré-v1.0) ; un fork demanderait une masse critique de contributeurs motivés qui n'existe pas encore clairement à ce stade.	Moyen – MIT permissif, mais communauté encore modeste (~1,8k ★) ; moins de garantie qu'une masse critique de contributeurs existe si un fork devenait nécessaire.	Moyen – MIT, communauté large et très active (14,6k ★, plus de 1 000 releases, rythme de publication quasi quotidien), mais fortement dépendant du moteur externe OpenCode : une évolution défavorable d'OpenCode affecterait aussi un éventuel fork d'OpenWork.	Élevé – communauté très large (81,3k ★) et écosystème actif ; en cas de dérive, une masse critique de contributeurs existe clairement.

Critère	Goose	VS Code + Cline	Eigent	Open Cowork	OpenWork	OpenHands
Avantages	~30k ★, gouvernance Linux Foundation (pérennité) ; compatible avec tout LLM dont le local (Ollama) ; workflows YAML partageables ("recipes").	Contrôle pas à pas explicite (approbation avant chaque action) ; ~58k ★ ; quasiment tous les fournisseurs de modèles dont le local ; développement soutenu (structure bien financée).	~14k ★ ; modèle-agnostique (Claude, GPT, Gemini, local) ; exécution locale ; fonctions entreprise (SSO, RBAC, audit) incluses.	Windows nativement supporté et gratuit ; Skills de génération de documents bureautiques utiles au cas d'usage 3 (Excel) ; multi-modèle large.	Traçabilité explicite et permissions action par action ; communauté large et active (14,6k ★, releases quasi quotidiennes) ; "ejectable" (pas de verrouillage interface).	Très grande communauté (81,3k ★) ; peut piloter plusieurs agents (Claude Code, Codex, Gemini) ; accès restreint à un dossier de projet en mode sandbox.
Inconvénients	Pensé pour un usage développeur (CLI/shell) ; UI Desktop plus simple mais peu guidée pour un public non-développeur.	Dépend de VS Code comme socle ; interface liée à un éditeur de code, peu adaptée aux profils généralistes.	Projet jeune (pré-v1.0), petite équipe (~8 pers.) ; architecture multi-agents plus complexe à configurer.	Organisation et mainteneurs anonymes ; communauté encore petite (1,8k ★) ; documentation limitée sur plusieurs critères.	Accès Windows payant – bloquant pour l'Audiar, majoritairement sous Windows ; dépend d'un moteur tiers (OpenCode).	Conçu pour des équipes d'ingénierie logicielle, peu adapté aux cas d'usage catalogue/Excel/QGIS ; mode "sans sandbox" à risque si mal configuré.

(pour tous les candidats) Aucun des 11 candidats n'a été testé concrètement avec les serveurs MCP déjà retenus dans ce document (crystaldba/postgres-mcp, haris-musa/excel-mcp-server, nkarasiak/qgis-mcp, filesystem) — reste un point aveugle à lever avant arbitrage final, quel que soit le candidat choisi.

SOLUTIONS ÉCARTÉES DU COMPARATIF

- **Roo Code** : un fork de Cline pourtant bien noté, archivé en mai 2026 (abandon de la maintenance) ;
- **Kilo Code** (fusion Cline + Roo Code) : écarté à ce stade car produit récent porté par une startup financée, sans le recul de maturité des autres candidats (à surveiller).
- **OpenClaw** : le plus gros projet trouvé en volume (361k+ ★), mais mauvaise catégorie : c'est un assistant personnel "toujours actif" connecté à des messageries (Telegram, Discord, WhatsApp), pas un agent de bureau en mode projet sur un dossier de travail local.
- **Suna** (société Kortix, Apache 2.0, ~19k ★, déjà rentable ~990k\$ de revenu fin 2025) : plateforme généraliste d'agents en conteneurs Docker isolés, positionnement plus proche d'un centre de commande d'automatisation d'entreprise que d'un agent tournant sur le poste local d'un chargé d'études avec accès direct à QGIS/Excel. Potentiellement à réexaminer plus tard.
- **AIDotNet/OpenCowork** et **Kuse Cowork** : deux autres projets se revendiquant "alternative Cowork", repérés mais non évalués en profondeur (traction/étoiles peu claires ou non trouvées).

Frameworks (développement sur-mesure)

Les frameworks regroupent des briques pouvant être utilisées pour construire soi-même un agent (interface, gestion de la mémoire, des droits, de la traçabilité...).

TABLEAU COMPARATIF

Framework	Repo	Licence	Auteur	Pérennité	Nature
LangChain	langchain-ai/langchain	MIT	LangChain Inc., startup financée par capital-risque (Sequoia notamment).	~128k-142k ★, 21-23,6k forks (le plus gros écosystème de la catégorie) – chiffres fluctuants selon la source consultée.	Framework généraliste pour chaîner appels LLM, outils et mémoire ; brique la plus bas niveau/la plus "boîte à outils" des six.
LangGraph	langchain-ai/langgraph	MIT	LangChain Inc. (même éditeur que LangChain, sous-produit de son écosystème).	~38k ★, 6,3k forks.	Orchestration d'agents sous forme de graphe d'états (boucles, branchements, reprise sur erreur) – la brique la plus proche de ce qu'il faudrait pour construire un "mode projet" sur mesure si cette voie était un jour retenue.
CrewAI	crewAIInc/crewAI	MIT	CrewAI Inc., startup financée (18M\$ levés, série A menée par Insight Partners).	~44-54k ★, ~5,9k forks.	Orchestration multi-agents par "rôles" (une équipe d'agents spécialisés se répartit une tâche).
AutoGen	microsoft/autogen	MIT	Microsoft Research.	~48-55k ★, 8,3k forks – mais projet en mode maintenance : Microsoft a fusionné AutoGen et Semantic Kernel dans un nouveau SDK unifié, Microsoft Agent Framework (microsoft/agent-framework , MIT, v1.0 sortie le 03/04/2026). AutoGen ne reçoit plus de nouvelles fonctionnalités, maintenance reprise par la communauté.	Framework multi-agents par conversations agent-à-agent ; à considérer comme un projet en fin de vie plutôt qu'un candidat d'avenir.

Framework	Repo	Licence	Auteur	Pérennité	Nature
Semantic Kernel	microsoft/semantic-kernel	MIT	Microsoft.	~27,9k ★. Même transition que AutoGen : absorbé dans Microsoft Agent Framework (v1.0, 03/04/2026) – pérennité du nom "Semantic Kernel" en tant que tel incertaine à moyen terme.	SDK "entreprise" pour intégrer des LLM dans une application .NET/Python/Java existante ; orienté intégration dans du code applicatif déjà en place plutôt que création d'un agent autonome.
Haystack	deepset-ai/haystack	Apache 2.0	deepset, société allemande fondée en 2018, financée par capital-risque.	~25k ★.	Framework historiquement orienté RAG/recherche documentaire (pertinent pour le cas d'usage 1 "catalogue"), support agent ajouté plus récemment et moins mature que les cinq autres sur ce plan.

LangGraph serait la brique la plus pertinente pour construire un mode projet sur mesure (le seul des six pensé pour l'orchestration d'agents à états, pas juste l'appel d'outils) ; AutoGen et Semantic Kernel sont à considérer comme des noms sortants, remplacés depuis avril 2026 par Microsoft Agent Framework.

SERVEURS MCP

Recensement des implémentations existantes évaluées pour chaque serveur MCP.

Liens utiles :

- doc MCP, LF Projects (The Linux Foundation projects) <https://modelcontextprotocol.io/docs/getting-started/intro>
- liste de MCP, glama.ai, <https://glama.ai/mcp/servers>
- liste de MCP, pulsemcp.com, <https://www.pulsemcp.com/servers>
- liste de MCP, mcp.so, <https://mcp.so>
- liste de MCP, <https://mcpservers.org>

mcp-qgis

TABLEAU COMPARATIF

Candidat	Repo	Notes	Pérennité
jjsantos01/qgis_mcp	https://github.com/jjsantos01/qgis_mcp	Implémentation d'origine. Plugin QGIS (socket server) + serveur MCP Python en face. Référencé dans le plugin officiel QGIS (plugins.qgis.org).	949 ★, 151 forks, 13 commits, aucune release taguée .
nkarasiak/qgis-mcp – candidat retenu (20/07/2026)	https://github.com/nkarasiak/qgis-mcp	Fork/variante avec capacités étendues (100+ outils MCP vs ~15 pour l'original).	Rythme de publication soutenu malgré moins d'étoiles que l'original (chiffres à revérifier depuis le relevé du 17/07/2026).
nic01asFr/QgisStreamMCP – repéré le 24/07/2026, non arbitré	https://github.com/nic01asFr/QgisStreamMCP	Approche différente des deux précédents : QGIS Desktop complet tourne dans un conteneur Docker (GUI accessible via noVNC), pas un plugin sur le poste. Point notable pour Audiar : catalogue intégré de 30+ jeux de données français en open data (BD TOPO, Admin Express, RPG, cadastre IGN, Corine Land Cover...) via un pipeline "smart_load" qui télécharge et indexe automatiquement (bien plus robuste qu'une connexion WFS live selon le projet). Ajoute aussi des "recettes" de workflows préconstruits (densité bâtie, risque inondation...) et des exports QField/Grist. À vérifier avant tout usage : fiabilité des outils structurés (point de vigilance déjà noté pour qgis-mcp en général), et cohérence avec le choix déjà fait d'un accès par poste plutôt que centralisé (decisions.md).	Très jeune et non éprouvé : 6 ★, 1 fork, un seul commit au dépôt, aucune release taguée – à surveiller mais pas encore comparable en maturité aux deux candidats ci-dessus.

Autres forks repérés le 24/07/2026 et écartés faute de traction ou de périmètre pertinent : *anitagraser/QGIS2OllamaMCP* (fork de *jjsantos01*, orienté Ollama, 7 ★, n'apporte pas de capacités supplémentaires notables) ; *dkudom/QGIS-MCP-Server* (projet ad hoc pour une collectivité au Ghana, 0 ★) ; *kicker315/deepseek_qgis_mcp* et *syauqi-uqi/qgis_mcp_modify1* (forks personnels sans indicateur de traction).

mcp-filesystem

TABLEAU COMPARATIF

Candidat	Repo	Notes	Pérennité
modelcontextprotocol/servers (src/filesystem)	https://github.com/modelcontextprotocol/servers/tree/main/src/filesystem	Implémentation de référence officielle (Node, @modelcontextprotocol/server-file-system). Contrôle d'accès par répertoires autorisés.	86,7k ★, 10,9k forks (chiffres du monorepo entier, pas du seul module filesystem), dernière release le 27/01/2026, maintenu par le MCP steering group/Anthropic – le plus pérenne des candidats recensés dans ce document.
cyanheads/filesystem-mcp-server	https://github.com/cyanheads/filesystem-mcp-server	Alternative communautaire avec fonctions avancées (recherche/remplacement, parcours d'arborescence, move_path / copy_path).	41 ★, 24 forks, 30 commits, 5 tags – projet individuel, activité modeste.

mcp-postgres

TABLEAU COMPARATIF

Candidat	Repo	Notes	Pérennité
crystaldba/postgres-mcp – retiré du statut "recommandé" le 25/07/2026, voir alerte ci-dessous	https://github.com/crystaldba/postgres-mcp	"Postgres MCP Pro". Accès lecture/écriture configurable + analyse de performance (le vrai différenciateur : tuning d'index, plans EXPLAIN , bilans de santé) – toujours les fonctions de tuning les plus abouties du tableau.	~2,4k ★ (source variable selon le moment de mesure), 259 forks – activité soutenue en apparence, largement contredite par l'alerte ci-dessous.
modelcontextprotocol/servers-archived (src/postgres)	https://github.com/modelcontextprotocol/servers-archived/tree/main/src/postgres	Ancienne implémentation de référence Anthropic, archivée depuis le 29/05/2025, non maintenue . Une faille d'injection SQL contournant la restriction lecture seule y a été documentée (Datadog Security Labs) – à éviter en production.	260 ★, 147 forks (dépôt archivé entier), 0 release, dépôt en lecture seule .

Candidat	Repo	Notes	Pérennité
googleapis/mcp-toolbox (ex- genai-toolbox) – repéré le 21/07/2026, non arbitré face au candidat retenu	https://github.com/googleapis/mcp-toolbox	Pas un serveur dédié Postgres : framework générique multi-bases (Postgres, MySQL, BigQuery, Spanner, SQLite...), maintenu officiellement par Google. Utilisable avec un Postgres auto-hébergé générique, pas seulement Cloud SQL/AlloyDB – pas de verrouillage Google Cloud. Deux modes : "prebuilt" (-- prebuilt=postgres , proche d'un accès SQL direct) ou configuration par fichier tools.yaml définissant des requêtes SQL paramétrées et figées à l'avance – approche plus contrainte/auditable que le SQL libre, cohérente avec la logique déjà retenue (mode restricted + rôle dédié en lecture seule). Outils de santé/perf riches (activité, locks, index, stats de requêtes, autovacuum, réplication...), comparables à postgres-mcp Pro ; pas d'équivalent identifié à l' EXPLAIN /recommandati on d'index automatique. Pas de mode read-only natif au niveau protocole – même recommandation de défense en profondeur par rôle Postgres. Windows compatible (binaire natif ou npx), transport stdio/HTTP.	~16 000 ★, ~1 600 forks, v1.0 stable depuis le 10/04/2026, rythme de release toutes les 2-3 semaines (jusqu'à v1.7 au 16/07/2026) – activité et pérennité fortes (Google, Apache 2.0).

Candidat	Repo	Notes	Pérennité
pgplex/pgconsole – repéré le 24/07/2026, à piloter en parallèle	https://github.com/pgplex/pgconsole	Angle différent de tous les candidats ci-dessus : ce n'est pas d'abord un serveur MCP mais une console SQL web self-hosted qui expose <i>en plus</i> un point d'accès MCP, avec le même modèle de droits pour un humain cliquant dans l'interface et pour un agent IA. L'agent ne voit jamais la chaîne de connexion : il obtient une identité propre (principal dédié), peut agir "pour le compte" d'un utilisateur en mode délégué (droits plafonnés à ceux de cet utilisateur, jamais au-dessus, perdus si l'utilisateur perd les siens), avec 7 droits fins (lecture, écriture, DDL, admin...) refusés par défaut sauf règle explicite, et un parseur SQL qui bloque les requêtes destructrices avant exécution (pas juste un flag "read-only" côté connexion, contournable comme chez l'ancien serveur Anthropic ci-dessus). Configuration versionnable façon GitOps (<code>pgconsole.toml</code>), binaire unique (npx ou Docker), pas de base séparée à administrer. Pertinent pour Audiar car directement structuré autour du besoin déjà identifié (accès lecture seule pour Charlotte sur BDSIG, traçabilité) plutôt que rajouté après coup.	Jeune : ~120 ★, Apache 2.0, premières releases récentes – pas encore le recul des autres candidats, mais modèle de gouvernance le plus abouti du tableau selon Bytebase (comparatif indépendant, source ci-dessous).

Candidat	Repo	Notes	Pérennité
bytebase/dbhub — candidat recommandé (25/07/2026)	https://github.com/bytebase/dbhub	Comme googleapis/mcp-toolbox, pas un serveur dédié Postgres mais multi-bases (Postgres, MySQL, MariaDB, SQL Server, SQLite) via une interface unique ; maintenu par Bytebase (éditeur d'une plateforme de gouvernance de bases de données, société établie). Se présente comme "zero-dependency, token-efficient" : seulement deux outils MCP principaux (<code>execute_sql</code> , <code>search_objects</code> avec divulgation progressive du schéma) plutôt qu'une longue liste d'outils, pour limiter la consommation de contexte. Garde-fous intégrés : mode lecture seule, limite de lignes, timeout de requête, tunnel SSH et TLS. Inclut aussi un "Workbench" web pour tester les requêtes/outils sans passer par un client MCP.	~2,9k ★, 242 forks, 513 commits, développement très actif, TypeScript, MIT.

Alerte sur l'ancien candidat recommandé (crystaldba/postgres-mcp) : selon un comparatif indépendant de Bytebase (30/06/2026, source ci-dessous), la société Crystal DBA a été rachetée par Temporal en septembre 2025 et le développement du serveur s'est **essentiellement arrêté depuis** — dernière release taguée (v0.3.0) en mai 2025, derniers commits significatifs jusqu'en janvier 2026 puis silence, 63 issues ouvertes dont une fuite de connexions en mode SSE non résolue, et du travail déjà mergé (transport streamable-HTTP) jamais publié en release ni dans l'image Docker. Point plus immédiat : **le mode par défaut est unrestricted (lecture/écriture complète, SQL arbitraire)** — tous les exemples du README utilisent ce mode par défaut, il faut explicitement passer `--access-mode=restricted` pour retrouver la restriction lecture seule déjà actée pour Audiar. Ses fonctions de tuning restent les plus abouties du tableau, mais l'état de maintenance ne permet plus de le recommander en l'état.

Arbitrage proposé (25/07/2026) : `bytebase/dbhub` remplace `crystaldba/postgres-mcp` comme candidat recommandé — maintenance active (contrairement à `crystaldba`), garde-fous simples et réellement opérants (lecture seule, limite de lignes, timeout), et flexibilité pour du SQL ad hoc adaptée au cas d'usage 2 (extraction depuis une table en base), sans le formalisme de requêtes prédéfinies de `googleapis/mcp-toolbox`. `pgplex/pgconsole` reste à surveiller en parallèle : c'est le seul candidat dont le modèle de droits (identité par agent, délégation "pour le compte de", plafonnée aux droits de l'utilisateur) correspond précisément au besoin de traçabilité par utilisateur déjà identifié pour Audiar — mais sa jeunesse (120 ★, premières releases) ne permet pas encore de le retenir seul. Cet arbitrage reste à confirmer par l'Audiar, pas encore testé en conditions réelles.

Point de vigilance ajouté après retour de Charlotte (24/07/2026) : dépendance runtime par candidat. Absent du tableau jusqu'ici, alors que c'est un paramètre de choix et pas un simple détail d'installation — en particulier tant que le poste d'exécution réel (poste de Charlotte aujourd'hui pour le test, futur serveur Merlin demain) n'est pas figé :

- `crystaldba/postgres-mcp` : Python, lancé via `uv / uvx` — déjà la brique retenue pour `qgis-mcp` (voir `docs/architecture/decisions.md`, section “pourquoi `uv / uvx`”). Aucune dépendance supplémentaire.
- `bytebase/dbhub` : nécessite Node.js ≥ 22.5 pour l'installation `npm / npx` — **dépendance nouvelle pour ce projet**, absente du poste de Charlotte (constaté le 24/07/2026 en testant). Alternative : image Docker officielle, qui réutilise Docker Desktop déjà installé pour Open WebUI (donc pas de nouvelle dépendance dans ce cas), mais rompt avec le pattern de lancement des autres serveurs MCP du projet (processus `uvx` direct sur l'hôte plutôt qu'un conteneur) — voir `servers/mcp-dbhub/README.md`.
- `googleapis/mcp-toolbox` : “binaire natif ou `npx`” (déjà noté dans le tableau ci-dessus) — même dépendance potentielle à Node.js si la voie `npx` est retenue ; à vérifier si le binaire natif Windows est réellement autonome.
- `pgplex/pgconsole` : “`npx` ou Docker” (déjà noté) — même alternative que `dbhub`.

Inconnu à ce stade, à vérifier avant de trancher : l'environnement réel du futur serveur Merlin (Docker et/ou Node.js déjà disponibles ?). Le choix Docker fait pour le test `dbhub` en cours (`servers/mcp-dbhub/`) est motivé par ce qui est disponible aujourd'hui sur le poste de Charlotte, pas par une préférence tranchée pour tout déploiement futur.

Mise à jour (24/07/2026) : la comparaison `dbhub` vs `crystaldba` se fait désormais en phase 1 du déploiement (poste individuel, identifiants personnels), pas en centralisé — voir `docs/architecture/decisions.md`, section “Modèle de déploiement de `mcp-postgres` : plan en 3 phases”. `dbhub` y tourne en transport `stdio` + `mcpo` (pas son mode HTTP natif, qui n'offre pas de chemin de connexion “Direct” personnel côté Open WebUI).

Sources :

- comparatif indépendant des MCP Postgres, Bytebase (30/06/2026), <https://www.bytebase.com/blog/top-open-source-postgres-mcp-servers/> - <https://github.com/bytebase/dbhub> - <https://github.com/pgplex/pgconsole>

mcp-excel

TABLEAU COMPARATIF

Candidat	Pérennité	Dépendance à Excel	Déploiement	Capacités	Mode hybride	Vigilance
haris-musa/excel-mcp-server	3,8k ★, 416 forks, développement actif (8 releases, dernière v0.1.8 le 12/04/2026)	Aucune – Python/openpyxl	Par poste, même montage que <code>mcp-qgis / mcp-postgres</code> : <code>uvx excel-mcp-server stdio</code> lancé en sous-processus par <code>mcpo (uvx mcpo --port <port> --api-key ... -- uvx excel-mcp-server stdio)</code> , exposé en OpenAPI ; Open WebUI – qui tourne dans un conteneur Docker (<code>decisions.mtd</code>) – l'atteint via <code>http://host.docker.internal:<port></code> , jamais <code>localhost</code> directement. Mode centralisé alternatif documenté par le projet (transport <code>streamable-http</code> , dossier <code>racine</code> <code>EXCEL_FILES_PATH</code> , chemins relatifs uniquement) : mais ce mode est lui aussi bâti sur FastMCP (variable <code>FASTMCP_PORT</code>), donc probablement soumis à la	Classeurs/feuilles (créer, copier, renommer, supprimer), lecture/écriture de cellules et formules avec validation de syntaxe, mise en forme complète (police, couleurs, bordures, fusion, format numérique, mise en forme conditionnelle), graphiques (ligne, barres, camembert, nuage de points, aires), tableaux croisés dynamiques (lignes/valeurs/colonnes/fonction d'agrégation), tableaux Excel structurés, insertion/suppression de lignes et colonnes, lecture des règles de validation de données existantes. Pas d'outil pour créer de nouvelles règles de validation, pas de VBA, pas de Power Query/DAX.	Non – écrit directement le fichier .xlsx ; échoue si le fichier est ouvert dans Excel (verrouillage, voir note ci-dessous).	Aucune pour un usage batch ; à écarter si le mode hybride est requis.

Candidat	Pérennité	Dépendance à Excel	Déploiement	Capacités	Mode hybride	Vigilance
			même protection anti-DNS-rebinding déjà rencontrée sur qgis-mcp-server (rejet du Host: host.docker.internal envoyé par Open WebUI en Docker, 421 Misdirected Request) – à vérifier avant d'y compter ; mcpo resterait sans doute nécessaire même dans ce mode. Ce mode centralisé suppose de toute façon que les fichiers soient rapatriés sur ce serveur, à l'écart du cadrage "fichiers locaux" (voir note ci-dessous).			

Candidat	Pérennité	Dépendance à Excel	Déploiement	Capacités	Mode hybride	Vigilance
negokaz/excel-mcp-server	953 ★, 113 forks, développement à l'arrêt (dernière release le 19/07/2025, ~1 an)	Aucune pour le mode fichier (Node/Go, multiplateforme) ; Excel + Windows requis pour le mode "Live editing"	Mode fichier, par poste (même montage <code>mcpo + host.docker.internal</code> que ci-dessus) : <code>npx --yes @negokaz/excel-mcp-server (stdio)</code> lancé en sous-processus par <code>mcpo</code> . Un déploiement centralisé serait techniquement envisageable (chemin fourni à chaque appel) mais sans mécanisme de restriction de dossier documenté équivalent à <code>EXCEL_FILES_PATH</code> – moins encadré que <code>harismusa</code> si cette option était explorée. Mode "Live editing" : impose d'être sur le même poste Windows que la session Excel (OLE local), par poste uniquement, même montage <code>mcpo</code> .	7 outils au total : lister les feuilles, lire des valeurs/formules avec pagination (utile sur de gros fichiers), écrire des valeurs et formules, créer une feuille ou un tableau Excel, copier une feuille, mettre en forme une plage (police, bordures, remplissage, format numérique/décimales). Pas de graphiques, pas de tableaux croisés dynamiques, pas de mise en forme conditionnelle dédiée, pas de validation de données, pas de VBA en amont (une version enrichie avec VBA/validation/mise en forme conditionnelle existe dans le fork <code>vKenjo</code> , non fusionnée, 2 ★ seulement).	Oui en principe, via un backend dédié (<code>excel_ole.go</code>) qui pilote Excel comme un objet COM plutôt que d'écrire le fichier – évite le verrouillage. Non détaillé au-delà de la mention dans le README, aucun retour d'usage trouvé.	Capacités nettement plus limitées que <code>harismusa</code> pour l'usage courant ; le mode hybride est la seule vraie raison de le considérer, et reste à valider en pratique.

Candidat	Pérennité	Dépendance à Excel	Déploiement	Capacités	Mode hybride	Vigilance
guillehr2/Excel-MCP-Server-Master	33 ★, 16 forks, aucune release taguée, mainteneur unique	Aucune	Par poste, même montage mcpo + host.docker.internal : npx @guillehr2/excel-mcp-server (stdio) – installe en plus, au premier lancement, des dépendances Python (fastmcp, openpyxl, pandas, numpy, matplotlib, xlsxwriter, xlrd, xlwt) : nécessite Node.js et Python sur la machine hébergeant le serveur, plus de composants à maintenir que haris-musa (Python seul via uvx).	Classeurs/feuilles (créer, ouvrir, sauvegarder, lister, ajouter/supprimer/renommer), écriture de cellules et formules, tableaux Excel avec mise en forme automatique, graphiques (50+ styles), tableaux de bord combinant plusieurs graphiques, filtrage de données, import/export CSV/JSON/SQL/PDF (seul candidat à le proposer). Tableaux croisés dynamiques annoncés dans la description du projet mais aucun outil dédié trouvé dans la documentation – à vérifier avant de s'y fier. Mise en forme conditionnelle non identifiée comme fonctionnalité distincte.	Non – même limite que haris-musa (écriture directe du fichier, verrouillage si Excel a le fichier ouvert).	Écart entre fonctionnalités annoncées et outils réellement documentés (TCD) ; traction très faible, un seul mainteneur.

Candidat	Pérennité	Dépendance à Excel	Déploiement	Capacités	Mode hybride	Vigilance
sbroenne/mcp-server-excel	185 ★, 32 forks, développement très actif (412 commits, 127 releases, dernière v1.8.68 le 28/05/2026)	Oui – Windows + Excel 2016 ou supérieur installé (pilotage via l'API COM)	Par poste uniquement, par nature (contrôle une instance Excel locale). Le serveur MCP (mcp-excel.exe) tourne en stdio – même besoin d'un mcpo local pour être joignable depuis Open WebUI en Docker (http://host.docker.internal:<port>) que les autres candidats. Distribution simplifiée par rapport au modèle uvx /terminal ouvert : exécutable autonome à ajouter au PATH, extension VS Code en un clic, ou paquet .mcpb pour Claude Desktop ; icône de suivi dans la barre système pour surveiller les sessions actives.	Le plus complet des quatre : classeurs/feuilles (y compris déplacement entre classeurs), plages (valeurs, formules, mise en forme, validation, protection), tableaux Excel, tableaux croisés dynamiques avec champs/membres calculés, graphiques (le plus riche : séries, tendances, étiquettes), mise en forme conditionnelle (règles + suppression), macros VBA (exécution et gestion de modules), Power Query et DAX/modèle de données (seul candidat à couvrir ce point), segments (slicers), plages nommées, connexions OLEDB/ODBC.	Oui – mode "Agent Mode" conçu pour ça : sur demande, l'IA affiche la fenêtre Excel qu'elle pilote et l'utilisateur observe les opérations en direct (barre d'état avec progression). Nuance : l'outil requiert l'accès exclusif au classeur – si l'utilisateur a déjà ouvert son propre fichier de son côté, il doit le fermer pour que l'outil le reprenne. Ce n'est donc pas une prise de main sur une session ouverte de manière autonome par l'utilisateur, mais une fenêtre Excel pilotée par l'outil et rendue visible sur demande.	Licence Excel Desktop à confirmer sur chaque poste (pas seulement Excel Online) ; le projet documente lui-même deux rapports de bugs de stabilité – fiabilité de l'automatisation COM à tester avant tout arbitrage définitif. Gouvernance du périmètre large (détail dans SECURITY.md / FEATURE_S.md du projet) : (1) VBA – bloqué par défaut par Excel lui-même ("Trust access to the VBA project object model" désactivé par défaut, aucune commande ne peut l'activer par programme) : vérifier que ce réglage reste désactivé sur les postes plutôt que de compter uniquement sur un filtrage côté mcpo . (2) Connexions OLEDB/ODBC (outil connection , 9 opérations) – peut créer

Candidat	Pérennité	Dépendance à Excel	Déploiement	Capacités	Mode hybride	Vigilance
						une connexion directe vers toute source pour laquelle un pilote est installé sur le poste (ex. PostgreSQL). Nuance (22/07/2026) : la vraie barrière de sécurité est le rôle/compte utilisé pour se connecter, pas l'outil qui initie la connexion – si l'accès de l'utilisateur à BDSIG est déjà en lecture seule (cas de Charlotte), une connexion créée depuis Excel hérite de cette même restriction, donc pas de risque d'écriture non autorisée dans ce cas. Deux raisons plus modestes de préférer tout de même centraliser l'accès BD via mcp-postgres : postgres-mcp impose une limite de temps d'exécution des requêtes que le pilote OLEDB/ODBC n'a pas (risque qu'une requête mal calibrée pèse sur une base partagée) ; et

Candidat	Pérennité	Dépendance à Excel	Déploiement	Capacités	Mode hybride	Vigilance
						la traçabilité serait éclatée entre deux chemins d'accès différents. Point annexe indépendant du niveau de droit : si des identifiants sont mémorisés dans la chaîne de connexion du classeur, un tiers qui ouvrirait ce fichier pourrait les réutiliser – risque de fuite d'identifiants à surveiller si le fichier circule. (3) Power Query (outil <code>powerquery</code>) – peut lui aussi atteindre des sources externes via son code M, indépendamment de l'outil <code>connection</code> ; non filtrable finement par <code>mcpo</code> (qui bloque un outil entier, pas des fonctions à l'intérieur) – risque résiduel à accepter ou trancher en écartant aussi cet outil. (4) Deux options optionnelles et désactivées par défaut, <code>formatMCode</code> / <code>formatData</code> , envoient le

Candidat	Pérennité	Dépendance à Excel	Déploiement	Capacités	Mode hybride	Vigilance
						code M ou les formules DAX à des services tiers externes (<code>powerqueryformatter.com</code> , <code>daxformatter.com</code>) pour reformatage – à ne jamais activer, cohérent avec la contrainte de souveraineté des données déjà actée.

COMMENTAIRES

Enjeu de la modification en direct

Un usage hybride étant attendu (utilisation de l'IA s'ajoute à l'utilisation des outils) et le contrôle des résultats produits étant un enjeu important, la comparaison a notamment porté sur la capacité des outils à interagir “en direct” dans les documents ouverts par les utilisateurs.

A cet égard, il faut noter que : - sous Windows, un fichier `.xlsx` ouvert dans Excel est verrouillé en écriture pour les autres processus ; une tentative d'écriture par `openpyxl` (ou toute bibliothèque équivalente manipulant le fichier directement) échoue avec `PermissionError` (`WinError 32`) tant que le fichier reste ouvert. Ainsi, les MCP de `haris-musa` et `guillehr2` ne peuvent pas offrir de mode hybride par construction, seule une lecture seule (`read_only=True`) reste possible fichier ouvert, jamais l'écriture. - ce fonctionnement n'est possible qu'avec le MCP tournant sur la machine de l'utilisateur – approche COM/OLE (`sbroenne` , `negokaz` en mode “Live editing”) –, pas de manière centralisée, qui ne permet que l'approche fichier (`haris-musa` , `negokaz` en mode normal, `guillehr2`)

Évaluation des candidats

`sbroenne` ressort comme le candidat le plus complet sur l'ensemble des besoins (formules toujours justes car évaluées par le vrai moteur Excel, TCD, mise en forme conditionnelle, charte graphique via les styles natifs) et le seul à offrir un vrai mode hybride (“Agent Mode”), avec une distribution plus simple pour 20 postes (exécutable autonome, icône barre système) que le modèle `uvx` /terminal.

Cependant, la fiabilité de l'automatisation COM en usage réel est à tester (le projet documente lui-même deux rapports de bugs de stabilité — à tester, pas à supposer). Par ailleurs, il permet beaucoup de manipulations (VBA, connexions OLEDB/ODBC externes), qui devraient être si besoin filtrées si l'Audiar estime que le risque est trop important.

`haris-musa` garde un intérêt comme repli plus simple et sans dépendance.

mcp-dataviz

Recherche effectuée le 24/07/2026.

Précision sur “create-viz” (25/07/2026) : ce n’est pas un serveur MCP mais un **“agent skill”** officiel Anthropic (dépôt [anthropics/knowledge-work-plugins](https://github.com/anthropics/knowledge-work-plugins), référencé sur mcpservers.org) — d’où l’absence de résultat lors de la recherche initiale, limitée aux dépôts MCP. La nature est fondamentalement différente des candidats du tableau ci-dessous : - pas de serveur à déployer ni d’outils exposés via le protocole MCP : le skill fournit un prompt structuré qui pousse Claude à **écrire et exécuter lui-même du code Python** (matplotlib/seaborn par défaut pour du statique “qualité publication” ; plotly si l’interactivité est demandée), à partir de données déjà disponibles dans la conversation (résultat de requête, DataFrame, fichier collé/importé). - sortie : fichier PNG enregistré localement (pas d’appel à un service cloud tiers, contrairement au comportement par défaut d’antvis/mcp-server-chart) — point favorable pour la contrainte de souveraineté déjà actée pour Audiar, à condition que Claude ait accès à un environnement Python avec ces bibliothèques (déjà le cas dans ce mode de travail). - inclut des règles de bonnes pratiques de dataviz (guide de choix du type de graphique selon la relation dans les données, palette daltonien-compatible, titres qui énoncent l’insight plutôt que la métrique, axes des ordonnées commençant à zéro pour les barres...) — une couche que les MCP du tableau ci-dessous n’offrent pas nativement, chacun se contentant de générer le graphique demandé.

Cette approche par skill est cohérente avec l’architecture déjà envisagée pour ce prototype (skills docx/xlsx/pptx déjà utilisés dans ce même mode de travail, cf. [decisions.md](#)) et pourrait être préférée à un MCP dédié si l’environnement d’exécution dispose déjà de Python — à recouper avec le choix de framework final (Open WebUI ne fait pas tourner de code Python arbitraire nativement, contrairement à ce mode de travail).

TABLEAU COMPARATIF (SERVEURS MCP)

Candidat	Repo	Capacité	Pérennité	Commentaire utile
antvis/mcp-server-chart	https://github.com/antvis/mcp-server-chart	26+ types de graphiques/diagrammes (barres, lignes, aires, camembert, boxplot, histogramme, radar, sankey, treemap, réseau, mindmap, organigramme, fishbone, logigramme, nuage de mots, venn, violon, entonnoir, dual-axes...) + tableau croisé/pivot ; outils de cartographie géographique séparés mais limités à la Chine (service AMap, non pertinents pour Audiar).	4,1k ★, 389 forks, 111 commits, MIT, maintenu par AntV (équipe visualisation d’Ant Group), 24 releases, dernière 0.9.10 (25/02/2026) – le plus starré et le plus mature du tableau.	Candidat le plus solide à date. Point de vigilance : par défaut, chaque image générée est rendue par un service cloud Alipay (antv-studio.alipay.com) – configurer la variable <code>VIS_REQUEST_SERVER</code> vers un déploiement privé (le projet fournit GPT-Vis-SSR pour ça) est nécessaire pour respecter la contrainte de souveraineté des données déjà actée pour Audiar. Windows compatible (<code>npx</code> ou Docker). Propose aussi un “skill” Claude Code dédié (sélection automatique du type de graphique).

Candidat	Repo	Capacité	Pérennité	Commentaire utile
mckinsey/vizro (sous-dossier vizro-mcp)	https://github.com/mckinsey/vizro	Portée différente et plus large : construit des tableaux de bord Vizro complets (multi-pages, filtres, cartes KPI, navigation) sur la base de Plotly/Dash, pas seulement des images de graphiques isolées – pertinent si l'objectif rejoint le cas d'usage 7 (construction d'un tableau de bord) déjà identifié plus haut dans ce document plutôt qu'un simple graphique ponctuel dans une conversation.	3,7k ★ (dépôt monorepo entier), 284 forks, 1 244 commits, Apache 2.0, maintenu par McKinsey, très actif (92 releases, dernière juin 2026).	À classer à part : ce n'est pas un générateur d'images de graphiques mais un outil de construction d'application de dashboard (sortie = app Python Dash à déployer, pas une image intégrée au chat). Le projet indique lui-même que Vizro-AI (approche précédente, génération de dashboard par LLM) est abandonné au profit de Vizro-MCP – bon signe de continuité, mais aussi le signe d'un outil encore jeune dans cette forme.
VisActor/vchart-mcp-server	https://github.com/VisActor/vchart-mcp-server	Graphiques interactifs via @visactor/vchart (bibliothèque backée par ByteDance) – principe proche d'antvis/mcp-server-chart mais écosystème distinct.	52 ★, 5 forks, MIT, TypeScript, actif (créé juillet 2025, dernier push mars 2026).	Nettement moins mature/adopté qu'antvis/mcp-server-chart ; à garder en solution de secours si ce dernier posait un problème bloquant (ex. dépendance au service cloud), non testé par ailleurs.
isaacwasserman/mcp-vegalite-server	https://github.com/isaacwasserman/mcp-vegalite-server	Génère des visualisations via la grammaire déclarative Vega-Lite (spécification ouverte, contrôle fin des graphiques), retourne une image PNG encodée en base64.	101 ★, 28 forks, Python, aucune licence déclarée dans le dépôt (point de vigilance juridique avant tout usage), dernier commit mai 2025 – développement apparemment à l'arrêt depuis plus d'un an.	Intéressant pour la rigueur de la grammaire Vega-Lite (portable, non liée à un service tiers) mais projet visiblement à l'arrêt et sans licence explicite – à écarter en l'état sauf signe de réactivation.

Candidat	Repo	Capacité	Pérennité	Commentaire utile
GongRzhe/Quickchart-MCP-Server – archivé, à écarter	https://github.com/GongRzhe/Quickchart-MCP-Server	Graphiques simples via l'API QuickChart.io (barres, lignes, camemberts, radar...).	159 ★, 44 forks, MIT, dernier push mai 2025, dépôt explicitement marqué "archived" par son auteur.	Traction correcte mais développement arrêté officiellement. Repose de toute façon sur le service tiers QuickChart.io (pas de rendu local) – aurait posé la même question de souveraineté qu'AntV, sans le niveau d'adoption.

Autres candidats repérés le 24/07/2026 et écartés faute de traction : SCKelemen/dataviz (2 ★, très récent, licence non standard "Other") ; TakanariShimbo/quickchart-mcp-server (2 ★, 0 fork) ; a-humphrey/plotly_mcp (1 ★, projet expérimental personnel). antvis/mcp-server-antv a aussi été écarté du tableau : ce n'est pas un générateur de graphiques mais un outil d'aide au développement (documentation contextuelle pour qui code avec les bibliothèques AntV), hors périmètre.

Sources :
- dépôts GitHub cités ci-dessus (README + métadonnées via l'API GitHub, relevé du 24/07/2026)

AUTRES OUTILS (OPEN WEBUI)

Liens utiles : - Open WebUI, <https://openwebui.com/search?query=web+search&sort=top&t=all&page=1>

Moteurs de recherche web

Recherche comparative pour la fonctionnalité de recherche web native d'Open WebUI (Settings > Web Search), relevée le 21/07/2026. Trois candidats évalués, tous nativement supportés comme provider Open WebUI.

Critère	Brave Search API	SearXNG	Tavily
Nature	Index propre (30+ milliards de pages), API commerciale hébergée par Brave.	Métamoteur auto-hébergé (Docker) : agrège DuckDuckGo, Qwant, Bing, Google, etc. – pas d'index propre.	API de recherche conçue dès l'origine pour l'usage agentique/LLM (endpoints "search" et "extract"), hébergée par Tavily.
Modèle économique / coût	Tier gratuit supprimé en février 2026 : désormais 5\$ de crédit mensuel offert (~1 000 requêtes), puis facturation à la carte ~4-5\$/1 000 requêtes. Carte bancaire requise dès l'inscription.	Gratuit à l'usage (aucune clé API, aucune carte bancaire) – seul coût : l'hébergement du service, déjà prévu puisqu'Open WebUI tourne déjà en interne.	1 000 crédits gratuits par mois, à vie, sans carte bancaire. Au-delà : 30\$/mois (4 000 crédits) ou 0,008\$/crédit en pay-as-you-go.
Estimation coût mensuel pour ~15 utilisateurs (~1 650 requêtes/mois)	~3-5 € (1 000 requêtes couvertes par le crédit, ~650 facturées en sus).	Gratuit (hors maintenance réseau, voir Fiabilité).	~5 € en pay-as-you-go (1 000 gratuites + 650 × 0,008\$) ou 30\$/mois si palier fixe choisi – moins granulaire que Brave.

Critère	Brave Search API	SearXNG	Tavily
Qualité / cohérence des résultats	Constante — index propre, pas dépendant de la tolérance d'un tiers au scraping.	Variable — dépend entièrement de ce que les moteurs sources laissent passer ce jour-là.	Bonne — reformate et enrichit les résultats spécifiquement pour un usage LLM (au-delà du simple classement web).
Adapté LLM/RAG	API dédiée ("LLM Context API") renvoyant des "smart chunks" pré-découpés en markdown ; outil de recherche déjà utilisé par Claude côté serveur.	JSON brut classique — nettoyage/reformatage à la charge du développeur avant injection dans le LLM.	Conçu nativement pour RAG : endpoints "search" + "extract" renvoyant du contenu déjà structuré pour un LLM.
Fiabilité opérationnelle	Bonne — infrastructure propre, pas de dépendance au scraping tiers.	Point faible documenté en 2026 : les moteurs sources (DuckDuckGo, Qwant) détectent le trafic agrégé comme du bot et renvoient CAPTCHA/erreurs ; une instance peut se dégrader progressivement sans hygiène réseau soignée (rate limiting, proxy sortant, réputation IP du serveur).	Service commercial dédié — aucun problème de CAPTCHA équivalent documenté dans les sources consultées.
Souveraineté / hébergement	Hébergé chez Brave (US), pas self-hosted.	100% auto-hébergé — cohérent avec l'infra déjà interne (Postgres, QGIS, Open WebUI), même si les requêtes sont in fine relayées aux moteurs sources.	Hébergé chez Tavily (US), pas self-hosted.
Compatibilité Open WebUI	Provider natif (brave).	Provider natif (searxng).	Provider natif (tavily).
Avantages	Qualité constante, faible latence, pensé pour l'usage LLM, coût prévisible et modéré au volume Audiar.	Gratuit, aucune clé API/carte bancaire, cohérent avec la logique self-hosted déjà en place pour le reste du projet.	Gratuit jusqu'à 1 000 requêtes/mois sans carte bancaire, sortie déjà formatée pour un LLM.
Inconvénients	N'est plus gratuit depuis février 2026 ; dépendance à un tiers commercial ; carte bancaire requise.	Fiabilité variable dans la durée (CAPTCHA), demande une maintenance réseau (rate limiting, proxy sortant) pour rester utilisable au quotidien.	Crédits gratuits dépassés dès ~1 000 requêtes/mois pour 15 utilisateurs ; palier payant (30\$/mois) moins granulaire que le pay-as-you-go de Brave.

Candidat identifié mais non évalué : Linkup, cité dans une comparaison tierce comme le provider "le plus précis" du marché — à creuser si Brave/SearXNG/Tavily s'avéraient insuffisants à l'usage.

Dataviz

Alternative aux serveurs MCP recensés plus haut (mcp-dataviz) : produire des graphiques directement via les fonctionnalités natives d'Open WebUI, en s'appuyant sur la capacité du modèle à écrire du code Python/JS plutôt que sur un outil de génération de graphiques préconstruit. Trois leviers natifs, non exclusifs entre eux.

TABLEAU COMPARATIF

Levier	Fonctionnement	Bibliothèques / capacités	Accès réseau	Infra à héberger	Avantages	Limites / vigilance
Code Interpreter – mode Pyodide	Python exécuté côté navigateur (WebAssembly) ; le modèle écrit le code, l'exécute, l'image du graphique s'affiche directement dans la conversation.	numpy, pandas, matplotlib, seaborn (+ scikit-learn, scipy, sympy pour l'analyse en amont).	Aucun – bac à sable isolé du réseau.	Aucune – natif, déjà inclus dans Open WebUI.	Zéro service supplémentaire à maintenir ; 100% local (tourne dans le navigateur de l'utilisateur) ; proche dans l'esprit du skill Claude create-viz documenté plus haut (le modèle écrit lui-même le code avec de bonnes pratiques), mais nativement intégré plutôt que packagé en skill.	Les données doivent déjà être présentes dans la conversation (collées, importées, ou renvoyées par un appel d'outil/Tool précédent) avant de pouvoir être tracées – ne permet pas seul d'interroger Postgres ou un fichier distant.
Code Interpreter – mode Jupyter	Depuis la v0.5.11, Open WebUI peut déléguer l'exécution à un serveur Jupyter externe plutôt qu'à Pyodide.	Mêmes bibliothèques Python que Pyodide, sans la limite du bac à sable.	Oui – le code exécuté peut atteindre le réseau, donc potentiellement une connexion directe à Postgres (ou autre source) depuis le code généré, sans passer par un Tool intermédiaire.	Un serveur Jupyter à déployer et maintenir (composant Docker supplémentaire).	Lève la limite d'accès réseau de Pyodide.	Composant de plus à administrer ; questions d'isolation/droits d'accès réseau à cadrer – pas de mode "restreint" documenté équivalent à ce qui a été retenu pour mcp-postgres .

Levier	Fonctionnement	Bibliothèques / capacités	Accès réseau	Infra à héberger	Avantages	Limites / vigilance
Artifacts	Le modèle génère une page HTML/JS/CSS autonome, rendue dans un iframe sandboxé directement dans le chat (CSP configurable via <code>IFRAME_CSP</code>).	Chart.js, D3.js, Three.js, SVG.	Non pertinent — rendu client, pas d'exécution Python côté serveur.	Aucune — natif.	Interactivité (survol, zoom, filtres) — plus proche d'un tableau de bord léger que d'un graphique statique ponctuel.	Demande que le modèle maîtrise la bibliothèque JS choisie ; pas de garde-fous de bonnes pratiques packagés, contrairement à un skill dédié type <code>create-viz</code> .

COMMENTAIRES

Montage envisageable pour Audiar

Combiner un Tool Python natif (pas MCP — cf. section précédente sur la distinction Tools/Skills) qui va chercher les données dans Postgres/Excel et les renvoie dans la conversation, puis Code Interpreter (Pyodide suffit si les données sont déjà récupérées) ou un Artifact pour la mise en forme visuelle. Avantages par rapport aux MCP dédiés du tableau `mcp-dataviz` : pas de dépendance à un service cloud tiers pour le rendu (point de vigilance relevé notamment pour `antvis/mcp-server-chart`, qui rend par défaut via un service Alipay) — cohérent avec la contrainte de souveraineté déjà actée pour Audiar. Pyodide tournant côté navigateur, ça reste local même sans toucher au serveur Docker.

Non arbitré à ce stade — reste à tester en conditions réelles (fiabilité du code généré par le modèle sans les garde-fous de bonnes pratiques qu'apporte un skill dédié type `create-viz`, richesse réelle par rapport aux 26+ types de graphiques tout faits d' `antvis/mcp-server-chart`).

Sources :
- [Python Code Execution — Open WebUI](#) - [Jupyter Notebooks — Open WebUI](#) - [Artifacts — Open WebUI](#)

SOURCES

Sources solutions d'orchestration

- <https://github.com/block/goose>
- <https://goose-docs.ai/>
- <https://github.com/Mintplex-Labs/anything-llm>
- <https://docs.anythingllm.com/mcp-compatibility/overview>
- <https://docs.anythingllm.com/agent/usage/file-system-agent>
- <https://github.com/open-webui/open-webui>
- <https://docs.openwebui.com/features/extensibility/mcp/>
- <https://github.com/open-webui/mcpo>
- <https://github.com/danny-avila/LibreChat>
- <https://www.librechat.ai/docs/features/mcp>
- <https://github.com/OpenHands/OpenHands>
- <https://github.com/cline/cline>
- <https://cline.bot/>
- <https://github.com/eigent-ai/eigent>
- <https://www.eigent.ai/blog/best-open-source-claude-cowork-alternatives-2026>
- <https://www.eigent.ai/blog/eigent-vs-claude-cowork>
- https://biggo.com/news/202511041923_open-webui-license-change-backlash
- <https://news.ycombinator.com/item?id=43901575>
- <https://github.com/VSCodium/vscodium>
- <https://vscodium.com/>
- <https://github.com/OpenCoworkAI/open-cowork>
- <https://opencoworkai.github.io/open-cowork/>
- <https://github.com/different-ai/openwork>
- <https://openworklabs.com/>
- <https://github.com/kortix-ai/suna> (candidat écarté, pour référence)
- <https://www.eigent.ai/blog/best-openclaw-alternatives-2026> (contexte OpenClaw, candidat écarté)
- <https://github.com/janhq/jan>
- <https://jan.ai/>
- <https://github.com/nomic-ai/gpt4all>
- <https://github.com/nomic-ai/gpt4all/blob/main/LICENSE.txt>
- <https://docs.openhands.dev/overview/introduction>
- <https://openhands.dev/>

Sources serveurs MCP

- https://github.com/jjsantos01/qgis_mcp
- https://plugins.qgis.org/plugins/qgis_mcp_plugin/
- <https://github.com/nkarasiak/qgis-mcp>
- <https://github.com/modelcontextprotocol/servers/tree/main/src/filesystem>
- <https://github.com/cyanheads/filesystem-mcp-server>
- <https://github.com/crystaldba/postgres-mcp>
- <https://github.com/modelcontextprotocol/servers-archived/tree/main/src/postgres>
- <https://securitylabs.datadoghq.com/articles/mcp-vulnerability-case-study-SQL-injection-in-the-postgresql-mcp-server/>
- <https://github.com/googleapis/mcp-toolbox>
- <https://googleapis.github.io/genai-toolbox/>
- <https://googleapis.github.io/genai-toolbox/reference/prebuilt-tools/>
- <https://cloud.google.com/blog/products/databases/managed-mcp-servers-for-google-cloud-databases>
- <https://github.com/haris-musa/excel-mcp-server>
- <https://github.com/negokaz/excel-mcp-server>
- <https://github.com/sbroenne/mcp-server-excel>
- <https://github.com/antvis/mcp-server-chart>
- <https://github.com/mckinsey/vizro>
- <https://github.com/VisActor/vchart-mcp-server>
- <https://github.com/isaacwasserman/mcp-vegalite-server>
- <https://github.com/GongRzhe/Quickchart-MCP-Server>
- <https://mcpservers.org/agent-skills/anthropic/create-viz>
- <https://github.com/anthropics/knowledge-work-plugins/tree/HEAD/data/skills/create-viz>

Sources moteurs de recherche web

- <https://docs.openwebui.com/features/chat-conversations/web-search/providers/brave/>
- <https://docs.openwebui.com/features/chat-conversations/web-search/providers/searxng/>
- <https://docs.openwebui.com/features/chat-conversations/web-search/providers/tavily/>
- <https://brave.com/learn/best-search-api-2026/>
- <https://www.implicator.ai/brave-drops-free-search-api-tier-puts-all-developers-on-metered-billing/>
- <https://costbench.com/software/ai-search-apis/brave-search-api/>
- <https://www.serverspan.com/en/blog/searxng-on-a-vps-how-to-run-private-search-without-getting-rate-limited-into-uselessness>
- <https://github.com/Fosowl/agenticSeek/issues/410>
- <https://docs.tavily.com/documentation/api-credits>
- <https://coldiq.com/blog/tavily-pricing>
- <https://www.linkup.so/blog/best-web-search-for-open-webui-setup-guide-and-provider-comparison>